

3DGS 在 Unity 游戏引擎中的应用边界调研与验证

——面向静态场景视觉代理、Web 轻量化部署与游戏交互融合的产品技术研究

张清安

字节跳动 · 抖音社交与互动团队

zhangqan@mail2.sysu.edu.cn

2026 年 6 月

摘要

3D Gaussian Splatting (3DGS) 作为一种新兴的显式辐射场表示方法，在实时新视角合成、真实场景重建与轻量化可视化方面表现出较强潜力。与传统 Mesh 管线相比，3DGS 不依赖连续拓扑建模，能够以多视角图像或点云为输入快速拟合复杂场景；但游戏引擎并不只需要“可渲染”的场景，还需要碰撞、光照、遮挡剔除、资源流送、编辑器工作流和跨平台部署等完整工程能力。因此，3DGS 在 Unity 游戏中的价值不应被简单理解为替代 Mesh，而应被放置在具体游戏开发链路中讨论。

本文基于个人在 Web 端 3DGS 展示、Unity 端 3DGS 渲染和 Three.js 游戏环境嵌入方面的实践，围绕“3DGS 在 Unity 游戏引擎中的应用边界”展开调研与验证。报告首先梳理 3DGS 与传统 Mesh、Imposter、LOD 和游戏引擎渲染管线之间的关系；随后总结当前 Demo 体系，包括 Azure 技术主页中的 Web 端实时 3DGS 展示、Unity 2022.3 + Compute Shader 的静态场景渲染、Three.js + Rapier 的网页游戏环境以及交互式 3D 小镇原型；进一步分析 Unity WebGL 打包失败、Shader 变体膨胀、渲染后端不兼容、资产管线缺失、动态交互弱和光照一致性不足等关键瓶颈。基于这些发现，本文提出“3DGS 视觉层 + Mesh 碰撞代理 + 分块流送 + Web 原生渲染兜底”的阶段化 Pipeline，并给出面向抖音社交与互动场景的产品化落地方向。

研究结论表明：在短期内，3DGS 更适合作为游戏中的静态复杂场景视觉代理，而不是完整替代 Mesh 管线；其优先落地点包括虚拟背景、虚拟展厅、空间漫游、加载关卡背景和轻交互小游戏场景。对于 Unity 框架，桌面端 DX12/Compute Shader 路线具备较好的演示价值；对于 Web 端轻量化产品，Three.js/WebGPU 或专用 3DGS 渲染器可能比 Unity WebGL 更适合作为近期落地路径。

关键词：3D Gaussian Splatting；Unity；WebGL；游戏开发工具；视觉代理；LOD；产品技术验证

目录

1 引言：从游戏内容生产压力到 3DGS 应用边界	4
1.1 游戏场景资产的生产与部署压力	4
1.2 3DGS 的机会：从重建技术到游戏视觉代理	4
1.3 本文研究命题	4
2 技术背景：Mesh、Imposter、LOD 与 3DGS	5
2.1 Mesh 管线的优势与边界	5
2.2 Imposter 与 LOD 的工程思想	5
2.3 3DGS 的基本表示与渲染特点	6
2.4 3DGS 不适合直接替代 Mesh 的原因	6
3 研究对象与实践环境	7
3.1 研究对象	7
3.2 硬件与软件条件	8
3.3 评估维度	8
4 当前 Demo 体系与实践结果	9
4.1 Web 端实时 3DGS 展示	9
4.2 Unity 引擎集成	9
4.3 Three.js 游戏环境	10
4.4 阶段性结果总结	10
5 核心技术瓶颈分析	10
5.1 渲染后端不兼容	10
5.2 Shader 变体膨胀与构建不可控	11
5.3 资产格式与编辑器工具链缺失	11
5.4 交互与物理系统缺失	12
5.5 动态场景与光照一致性	12
5.6 包体、加载与移动端约束	12
6 可行性 Pipeline 设计	13
6.1 总体架构	13
6.2 阶段一：静态场景展示	15
6.3 阶段二：半动态游戏环境	15
6.4 阶段三：Web 原生轻量化产品	15
6.5 阶段四：长期能力建设	15

7 产品化价值与抖音场景落地点	16
7.1 从技术能力到产品价值	16
7.2 短视频虚拟背景	16
7.3 虚拟展厅与空间社交	16
7.4 小游戏场景生成	16
7.5 应用优先级矩阵	16
8 交付	17
8.1 近期交付	17
9 风险分析与后续研究	19
9.1 主要风险	19
9.2 后续研究方向	19
10 结论	19

1 引言：从游戏内容生产压力到 3DGS 应用边界

1.1 游戏场景资产的生产与部署压力

近年来，游戏内容生产正在同时面对两个方向的压力。一方面，玩家和创作者对场景真实感、空间沉浸感和内容规模提出了更高要求；另一方面，中小团队、UGC 创作者和轻量小游戏生态又需要更低成本、更自动化、更易分发的生产工具。传统游戏场景通常以 Mesh 为核心资产表达，围绕建模、UV 展开、纹理制作、材质配置、碰撞体构建、LOD 生成、光照烘焙和平台适配形成了成熟但复杂的工业管线。

Mesh 管线的优势非常明确：拓扑清晰、可编辑性强、可以直接参与物理碰撞、导航寻路、动画绑定和材质渲染，因此仍然是游戏主资产的核心形式。但在许多场景中，开发者并不一定需要完整的几何可编辑性。例如远景建筑群、复杂自然环境、扫描式真实空间、虚拟展厅背景和加载关卡视觉代理等，更关心的是“足够真实、生成成本低、运行开销可控”。这些场景为 3DGS 进入游戏开发链路提供了切入点。

1.2 3DGS 的机会：从重建技术到游戏视觉代理

3D Gaussian Splatting (3DGS) 由 Kerbl 等人提出后，迅速成为实时辐射场渲染与新视角合成领域的重要技术路线。与 NeRF 使用隐式神经网络表示场景不同，3DGS 使用大量显式三维高斯基元表示空间，每个高斯基元通常包含位置、尺度、旋转、颜色球谐系数和不透明度等参数。渲染时，高斯基元被投影到屏幕空间并进行排序混合，从而实现实时高质量渲染。

从游戏开发视角看，3DGS 的价值不在于“直接替代 Mesh”，而在于提供一种新的视觉代理能力。它能够较低门槛地从多视角图像生成高真实感场景表示，使真实空间、复杂建筑和不规则环境可以快速进入交互式应用。与传统 Imposter、Billboard 或贴图式背景相比，3DGS 可以提供更连续的视角变化；与完整 Mesh 资产相比，它降低了复杂场景重建和优化的人工成本。

1.3 本文研究命题

本文不将 3DGS 视为传统游戏资产的完整替代方案，而将其定位为一种面向静态复杂场景的**视觉代理技术**。本文关注的问题是：在 Unity 游戏引擎和 Web 端交互环境中，3DGS 能够承担哪些场景表达任务，不能承担哪些任务；如果要把它引入游戏开发工具链，近期可落地的工程路径是什么。

具体而言，本文围绕以下问题展开：

- (1) **技术边界**：3DGS 在 Unity 中可以完成什么级别的实时渲染、交互和部署？
- (2) **工程瓶颈**：为什么 Unity Editor 能跑通的 3DGS 场景难以直接导出为 WebGL 产品？
- (3) **产品价值**：3DGS 对抖音社交、小游戏和 AIGC 游戏开发工具链可能产生什么帮助？
- (4) **近期路径**：在有限硬件资源和实习阶段时间约束下，怎样设计一个可展示、可解释、可迭代的 Demo？

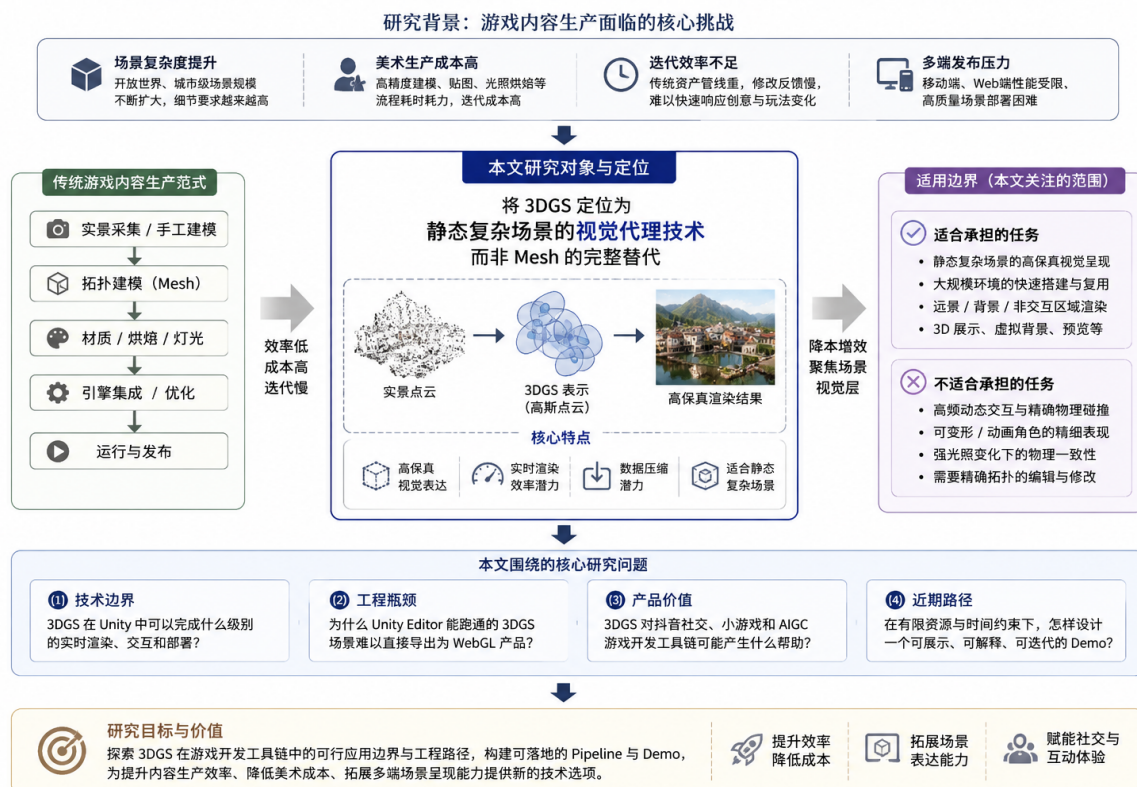


图 1: 从游戏内容生产压力出发，将 3DGS 定位为静态复杂场景视觉代理，而非 Mesh 的完整替代。

2 技术背景：Mesh、Imposter、LOD 与 3DGS

2.1 Mesh 管线的优势与边界

在游戏工业中，Mesh 是最核心、最成熟的几何表达方式。Mesh 以顶点、边和面片表示物体表面，结合纹理、材质、骨骼、碰撞体和导航网格，形成完整的游戏资产工作流。对于角色、道具、可破坏物体和强交互关卡，Mesh 的优势几乎不可替代。

但 Mesh 的生产成本同样很高。复杂真实场景通常需要经历采集、重建、拓扑清理、材质重建、LOD 制作、碰撞体简化、光照烘焙和平台性能调优等环节。尤其对于扫描式真实空间或开放场景远景，开发者往往只需要其视觉外观，而不需要每一处几何细节都可交互。此时，完整 Mesh 管线可能出现“表达能力过剩、生产成本过高”的问题。

2.2 Imposter 与 LOD 的工程思想

现代游戏引擎为了在有限硬件资源下表现大规模场景，通常依赖三类核心机制：LOD、Culling 和 Streaming。LOD 根据距离和重要性动态切换细节层级，Culling 剔除当前不需要渲染的对象，Streaming 则按需加载和卸载资源。Imposter 可以被理解为这些优化思想的一种具体表达：在某些上下文中，用足够便宜的替身表示复杂对象。

传统 Imposter 包括 Billboard、Flipbook、Mesh Card、Octahedral Imposter 等形式。它们适合表达远处植被、建筑外观或静态装饰，但在视角变化较大或几何结构复杂时容易出现失真。3DGS 的潜在优势正在于此：它仍然是三维空间中的显式点状/椭球状表示，可以提供比二维贴

图 Imposter 更自然的视角连续性，同时又避免完整 Mesh 重建所需的拓扑约束。

因此，本文将 3DGS 放在 Imposter 和 LOD 的工程语境下理解：它不是完整游戏资产，而是一种可能用于远景、背景、扫描空间和低交互区域的高真实感视觉替身。

2.3 3DGS 的基本表示与渲染特点

一个典型的 3DGS 场景由大量高斯基元组成。每个高斯基元可以被理解为空间中的一个带颜色和透明度的椭球体，其主要参数包括：

- **中心位置** $\mu \in \mathbb{R}^3$ ：控制高斯基元在三维空间中的位置；
- **尺度与旋转**：通常通过缩放向量与四元数表示协方差结构，决定椭球体形状；
- **颜色参数**：常使用球谐系数表达视角相关颜色；
- **不透明度**：控制高斯基元对图像的贡献和混合权重。

3DGS 的渲染特点可以概括为三点：第一，显式基元使其比 NeRF 更容易做实时光栅化；第二，高斯基元半透明，需要根据视角进行排序与混合；第三，颜色和光照通常被一并编码在训练结果中，因此与游戏引擎中的动态光照和阴影系统存在天然不一致。

2.4 3DGS 不适合直接替代 Mesh 的原因

从视觉效果看，3DGS 能够生成非常真实的新视角图像；但从游戏引擎角度看，它仍有明显短板：

- (1) **缺乏连续表面**：高斯点云没有显式拓扑，不适合直接承担精确碰撞、拾取和导航；
- (2) **半透明排序开销**：大量透明基元使遮挡剔除和渲染排序更加复杂；
- (3) **动态编辑困难**：移动、破坏、变形或替换局部物体需要重建或复杂的在线更新；
- (4) **光照解耦不足**：场景通常烘焙了训练时光照，难以自然响应游戏动态光源；
- (5) **资产工具链缺失**：Unity 尚未将 3DGS 作为原生资产类型管理，导入、压缩、LOD 和打包都依赖社区方案。

因此，本文的基本判断是：3DGS 在游戏中的近期价值是**补充型视觉资产**，而不是**通用型几何资产**。

表 1: Mesh、传统 Imposter 与 3DGS 在游戏场景中的能力对比

维度	Mesh	传统 Imposter	3DGS
几何与编辑	拓扑明确，可编辑性强	多为贴图替身，可编辑性弱	缺少连续拓扑，编辑能力弱
视角表现	自由视角，几何稳定	依赖采样角度，视角连续性有限	支持自由视角，适合漫游观察
生成成本	需建模、清理、材质制作	需离线烘焙多视角贴图	多视角图像即可训练，采集门槛较低
交互支持	原生支持碰撞、导航与物理	通常需要额外代理	需要额外 Mesh Collider 或导航代理
光照适配	可接入 PBR 与动态光照	多为预烘焙光照	训练光照固化，动态光照适配有限
典型用途	角色、道具、关卡主体	远景、植被、低视角变化物体	静态复杂背景、扫描空间、远景视觉代理

3 研究对象与实践环境

3.1 研究对象

本文基于已搭建的 3DGS 相关 Demo 与实验记录进行扩展分析。当前实践对象包括四条线：

(1) Web 端 3DGS 实时展示

基于 TriSplat、Three.js 与 WebGL 构建网页端 3DGS 展示页面，并部署在 Azure 个人技术主页中，用于验证浏览器环境下 3DGS 场景的轻量化展示能力。

(2) Unity 端 3DGS 渲染

基于 Unity 6000、Compute Shader 以及 GaussianSplats3D / UnityGaussianSplatting 类方案，验证桌面端 3DGS 场景的编辑器运行、实时渲染与工程集成能力。

(3) Three.js 游戏环境

基于 three-player-controller、Three.js 与 Rapier 物理引擎，构建包含角色移动、射击、多人联机和 3DGS 场景嵌入能力的 Web 端交互式游戏环境。

这些实践并不是为了证明 3DGS 已经可以完整替代 Unity 游戏 workflow，而是用于拆解不同层次的可行性：能不能展示、能不能交互、能不能打包、能不能产品化。

3.2 硬件与软件条件

表 2: 实验与 Demo 环境

项目	配置或说明
主要硬件	RTX 4060 Laptop GPU, 约 8GB VRAM; 另考虑单卡 24GB 作为可接受上限
验证环境	Unity 6000 LTS, URP, DX12 / Compute Shader 路线 / Three.js、WebGL
游戏交互	three-player-controller、Rapier 物理引擎、基础角色控制与场景漫游
测试资产	Outdoor.splat, 约 120 万高斯点, 约 36.6MB; DL3DV / RE10K 示例场景
主要目标	验证 3DGS 在 Web 展示、Unity 渲染、网页游戏交互和产品 Demo 中的适用边界

3.3 评估维度

表 3: 3DGS 游戏化应用的可行性评估维度

评估维度	关注问题
视觉质量	自由视角漫游是否自然, 边缘伪影、空洞区域和漂浮高斯点是否明显。
运行性能	桌面端和 Web 端帧率是否达到基本交互要求, 渲染开销是否可控。
包体与加载	场景数据、代码体积和构建产物是否适合 Web 页面、小游戏或轻量化应用分发。
交互兼容	是否能够支持角色控制、碰撞检测、射线拾取、视角切换和基础游戏逻辑。
工程管线	是否能够形成稳定的导入、压缩、渲染、调试、打包和发布流程。
产品价值	是否能够在创作工具、虚拟空间、小游戏或社交互动场景中形成差异化体验。

4 当前 Demo 体系与实践结果

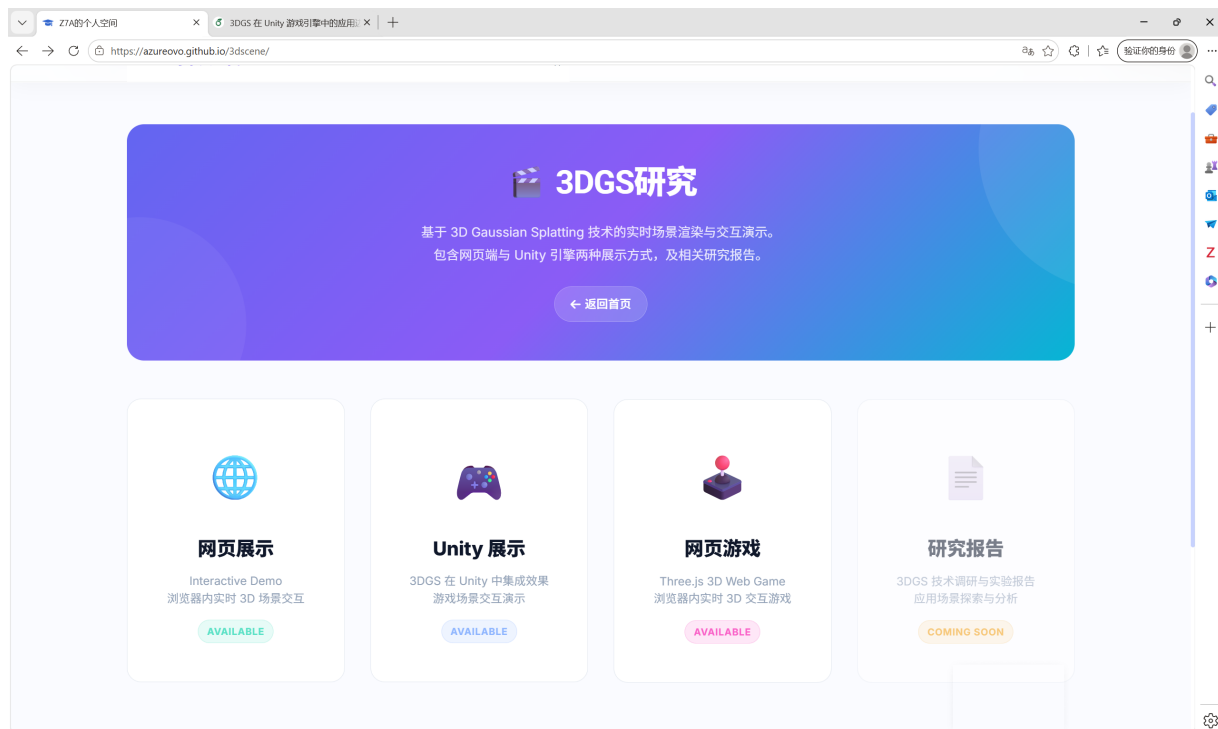


图 2: 当前 Demo 体系: Web 端实时展示、Unity 端渲染和 Three.js 游戏环境共同构成可行性验证矩阵。

4.1 Web 端实时 3DGS 展示

Web 端展示主要用于验证 3DGS 是否能够以较低接入成本进入浏览器环境。该部分基于 TriSplat 引擎、Three.js 与 WebGL 实现, 支持实时漫游、多场景切换、第一人称视角和角色控制。Demo 部署在 Azure 个人技术主页, 具备较好的可分享性和展示性。

从结果看, 浏览器端 3DGS 在 RTX 4060 Laptop 上能够保持 30–60 fps 的交互帧率。这说明对于静态展示、作品集、虚拟展厅和技术主页等场景, 3DGS 的 Web 端落地门槛已经相对可控。其优势在于无需安装客户端, 用户可以直接通过链接进入三维场景; 不足在于复杂场景的加载时间、移动端兼容性和输入控制体验仍需优化。

4.2 Unity 引擎集成

Unity 端集成主要用于验证 3DGS 是否能够进入传统游戏引擎 workflow。当前方案基于 Unity 2022.3、URP、DX12 Compute Shader 与 GaussianSplats3D/UnityGaussianSplatting 类渲染路线。测试资产为 Outdoor.splat, 约 120 万高斯点, 数据体积约 36.6MB。

在 Unity Editor 桌面端环境中, 3DGS 场景可以完成实时渲染, 并达到约 55–70 fps 的表现。这说明在桌面端本地运行条件下, 3DGS 的渲染性能并非主要问题。真正困难在于 Unity 的资产管线、构建管线和 WebGL 部署链路。

Unity WebGL 是本研究中暴露问题最多的环节。构建过程中出现 Shader 变体数量膨胀, 日志显示类似 `preparing variants (1510M / 72477M)` 的进度信息。这意味着 URP 与相关

Shader Keyword 组合产生了极大规模的变体空间，导致编译时间、构建体积和资源管理失控。

即使尝试启用 Shader Stripping、Strip Engine Code、IPreprocessShaders 预处理回调等优化手段，仍然难以得到可用于在线发布的 WebGL 包。更根本的问题是：Unity 端 3DGS 渲染方案依赖 Compute Shader 或 DX12/Vulkan 风格的 GPU 计算能力，而 WebGL 基于 OpenGL ES 子集，对 Compute Shader 支持有限。这使得“Unity Editor 能跑”并不等价于“Unity WebGL 能发布”。

4.3 Three.js 游戏环境

为了绕开 Unity WebGL 的后端限制，本研究同时搭建了 Three.js 游戏环境。该 Demo 基于 three-player-controller、Three.js 和 Rapier 物理引擎，实现角色控制、多人联机、射击模式和 3DGS 场景嵌入。这条线验证了一个重要判断：若目标是 Web 端轻交互产品，Web 原生渲染路线可能比 Unity WebGL 更适合近期落地。

Three.js 路线的优势是轻量、可控、方便部署，且与 Web 端 3DGS 渲染器天然兼容；不足是缺少 Unity 那样成熟的编辑器、资源管理和团队协作能力。因此，它更适合技术 Demo、虚拟空间原型、轻量小游戏或活动页，而不是复杂商业游戏开发。

4.4 阶段性结果总结

表 4: 当前 Demo 体系的功能与限制

Demo 类型	已验证能力	主要限制	适合用途
Web 端 3DGS 展示	浏览器实时渲染、多场景切换、链接分享	移动端性能和加载仍需优化	技术主页、作品展示、虚拟展厅
Unity 端 3DGS	桌面端实时渲染、可进入编辑器环境	WebGL 构建困难，缺少原生资产管线	本地 Demo、桌面应用、技术验证
Three.js 游戏环境	角色控制、物理引擎、轻量交互	编辑器和团队协作弱于 Unity	网页小游戏、轻量空间社交

5 核心技术瓶颈分析

5.1 渲染后端不兼容

3DGS 的高效渲染通常依赖 CUDA、Vulkan、DX12 Compute Shader 或 WebGPU 等具备较强并行计算能力的后端。Unity 桌面端可以通过 Compute Shader 实现高斯排序、投影与混合，但 WebGL 目标平台受限于 OpenGL ES 能力，难以直接承载这类管线。因此，Unity 端可运行的 3DGS 渲染器并不能直接迁移到浏览器发布。

这一问题提示我们，在产品方案选择上必须区分“引擎 Demo”和“线上可分发产品”。若目标是技术展示或本地桌面端交付，Unity 路线具有可行性；若目标是抖音活动页、轻量小游戏或 Web 端社交空间，则应优先考虑 Web 原生 3DGS 渲染器。

5.2 Shader 变体膨胀与构建不可控

Unity URP 在支持多光源、阴影、后处理、雾效以及多平台适配时，会根据不同渲染特性和关键字组合生成大量 Shader Variant。对于普通 Unity 项目而言，Shader Variant 本身已经可能显著拉长构建时间；而在引入 3DGS 渲染器之后，项目还需要额外维护自定义的高斯点渲染 Shader、排序逻辑和数据读取路径，使变体空间进一步扩大。构建日志中出现百亿级变体计数，说明问题已经不只是“构建速度较慢”，而是当前完整 URP 工程与 3DGS 自定义渲染管线叠加后，已经超出了轻量 Web 产品可控的打包范围。

针对这一问题，短期内可以从工程裁剪的角度进行缓解。首先，应尽量固定目标渲染管线，减少不必要的 URP Feature，避免在 WebGL 版本中保留桌面端才需要的阴影、后处理和复杂光照选项。其次，可以通过 Shader Stripping 删除无用 Keyword 和平台分支，只保留 Demo 所需的最小渲染路径。对于 3DGS 渲染器本身，也应尽可能将其独立为一个最小 Shader 集合，避免与完整 URP 游戏工程的材质系统、光照系统和后处理系统发生过度耦合。更合理的做法是不要直接将完整 Unity URP 工程导出为 WebGL 产品，而是为 Web 端单独维护一个轻量化渲染版本，将 3DGS 展示、基础相机控制和必要交互逻辑拆分出来，形成面向浏览器部署的最小可用工程。

不过，上述优化主要解决的是构建体积和打包复杂度问题，只能在一定程度上缓解 Shader Variant 膨胀带来的工程负担，并不能从根本上消除 Unity 桌面渲染路线与 WebGL 后端能力之间的差异。尤其是当 3DGS 渲染器依赖 Compute Shader、GPU Buffer 和复杂并行排序时，WebGL 平台本身的图形 API 能力限制会成为更深层的瓶颈。因此，从近期落地角度看，更可行的策略不是强行将 Unity 端 3DGS 工程完整搬到 WebGL，而是将 Unity 作为桌面端编辑与验证环境，将 Web 端展示交给 Three.js / WebGL 原生路线，以两条工程路线分别服务“编辑器验证”和“轻量化分发”。

5.3 资产格式与编辑器工具链缺失

传统 Unity 资产已经形成了较为成熟的工程化工作流。开发者可以通过 Importer 将模型、贴图、动画等资源导入项目，再借助 Prefab、Material、Collider、Animator 和 Build Pipeline 完成编辑、复用、交互配置和跨平台构建。也就是说，Mesh 资产并不只是一个几何文件，而是能够被 Unity 编辑器理解、管理和调度的完整资产类型。

相比之下，3DGS 资产通常以 .ply、.splat、.spz 或自定义二进制格式存在。Unity 默认并不理解这些文件中高斯点的位置、尺度、旋转、颜色、不透明度和球谐系数等语义，因此它们更接近“外部查看器数据”，而不是 Unity 原生资产。这种语义缺失会带来一系列工程问题：首先，3DGS 场景无法像 Mesh 一样在 Scene View 中直接编辑局部结构，开发者很难对某一片区域、某一组高斯点或某一处视觉伪影进行精细修改；其次，3DGS 本身不包含连续拓扑，因而无法原生成碰撞体、导航网格和遮挡代理，角色移动、射线拾取、物理碰撞等游戏逻辑仍然需要额外的 Mesh 或简化几何来支撑；再次，3DGS 的颜色和光照往往已经被训练过程固化，难以自动参与 Unity 的材质、阴影和动态光照系统；最后，由于缺少标准化资产描述，3DGS 场景也难以稳定接入 Addressables、AssetBundle 或增量加载流程，调试、性能分析和跨平台发布工具链都不够完善。

因此，如果希望 3DGS 在游戏开发中形成可规模化的应用方案，就不能只把它看作一个能

够被渲染的场景文件，而需要将其从“查看器资产”提升为“可管理资产”。这意味着工具链至少需要包含 3DGS 导入器、格式转换与压缩模块、编辑器预览器、LOD 或重要性排序生成器，以及运行时加载和卸载模块。只有当 3DGS 能够被 Unity 像管理 Mesh、Texture 或 Prefab 一样管理时，它才可能真正进入游戏开发工具链，而不是停留在单个 Demo 或展示页面层面。

5.4 交互与物理系统缺失

3DGS 本身没有连续表面，因此不能直接用于精确碰撞。如果用户在 3DGS 场景中行走，角色控制器需要一个独立的碰撞代理，例如低模 Mesh、简化地形或手工布置的 Collider。本文建议采用“视觉层与物理层分离”的架构：3DGS 负责高真实感画面，Mesh Collider 负责碰撞、导航和触发器。

这种架构牺牲了部分几何一致性，但符合游戏工程常见做法。许多游戏本身也不会使用视觉 Mesh 直接作为碰撞体，而是使用简化 Collider。因此，对于静态空间漫游类场景，3DGS + Collider Proxy 是近期最可行的组合。

5.5 动态场景与光照一致性

3DGS 在静态场景表现较好，但动态交互仍是难点。角色、道具、机关、特效和破坏系统都需要场景能够响应时间变化。4DGS、PhysGaussian 等研究尝试解决动态与物理问题，但其训练和推理成本仍不适合轻量游戏产品。

另一个重要问题是光照一致性。3DGS 的颜色通常包含训练时环境光照，游戏中新增动态光源后，3DGS 场景不会像 Mesh/PBR 材质一样自然响应。解决方向包括光照解耦、逆向渲染、局部 relighting 或把 3DGS 限制在预烘焙视觉层中。结合本人过往逆向渲染研究经验，光照解耦可以作为长期探索方向，但不应成为近期 Demo 的核心依赖。

5.6 包体、加载与移动端约束

Web 端和小游戏生态对包体和首屏加载非常敏感。一个 30-40MB 的 3DGS 场景在桌面浏览器中可以接受，但若叠加多个场景、角色模型、音频、物理库和交互脚本，总包体容易超过百 MB。对于抖音内嵌场景或小游戏来说，必须采用分块加载、压缩、低清首屏和渐进式细化策略。

表 5: 核心瓶颈、影响范围与建议对策

瓶颈	影响	建议对策
渲染后端不兼容	Unity WebGL 难以复用桌面端 Compute Shader 管线	桌面端用 Unity, Web 端改用 Three.js/WebGPU 原生渲染器
Shader 变体膨胀	构建时间、包体和稳定性不可控	极简 URP 配置、Shader Stripping、独立渲染工程
资产管线缺失	难以规模化导入、编辑和发布	设计自定义 Importer、压缩工具和运行时 Loader
碰撞与导航缺失	无法直接成为可玩关卡	使用低模 Mesh Collider 作为物理代理
动态与光照弱	不适合强交互和动态破坏	限定为静态视觉代理, 动态物体仍用 Mesh
包体和加载压力	影响 Web/小游戏首屏体验	SPZ/量化压缩、分块流送、LOD 渐进加载

6 可行性 Pipeline 设计

6.1 总体架构

基于前文对渲染能力、WebGL 打包、资产管线和交互兼容性的分析, 本文提出一种面向近期 Demo 和产品验证的混合式 Pipeline。该方案不试图将 3DGS 直接替代传统游戏资产, 而是将其嵌入 Unity 或 Web 端交互应用中, 作为静态复杂场景的视觉表达层; 碰撞、导航、触发器和动态交互仍由传统 Mesh 与游戏逻辑系统承担。整体流程可以概括为: 首先由多视角图像、扫描数据或公开数据集生成 3DGS 场景资产; 随后对高斯点云进行压缩、裁剪、格式转换和 LOD 处理; 在运行时, 3DGS 负责高真实感视觉呈现, Mesh Proxy 负责游戏逻辑和物理交互, 动态对象仍沿用传统角色、道具、粒子和动画系统; 最后根据目标平台选择 Unity 桌面端、Web 原生渲染或轻量化互动页面进行发布。

面向游戏化应用的 3DGS 混合 Pipeline

将 3DGS 作为视觉表达层，与 Mesh 代理、动态对象和资源管理协同工作



图 3: 面向游戏化应用的 3DGS 混合管道。

该 Pipeline 的核心思想是“视觉与逻辑分离”。3DGS 并不直接参与复杂游戏逻辑，而是提供真实感强、构建成本较低的静态视觉场景；传统 Mesh 系统则继续承担 Unity 和 Web 游戏框架中已经成熟的物理、交互和资源管理能力。通过这种分层方式，可以在不强行解决动态 3DGS、光照解耦和完整资产编辑等长期问题的前提下，先构建一个可展示、可解释、可迭代的近期 Demo。

推荐 Pipeline：3DGS 负责视觉表达，Mesh Proxy 负责碰撞和游戏逻辑，动态对象继续使用传统游戏资产

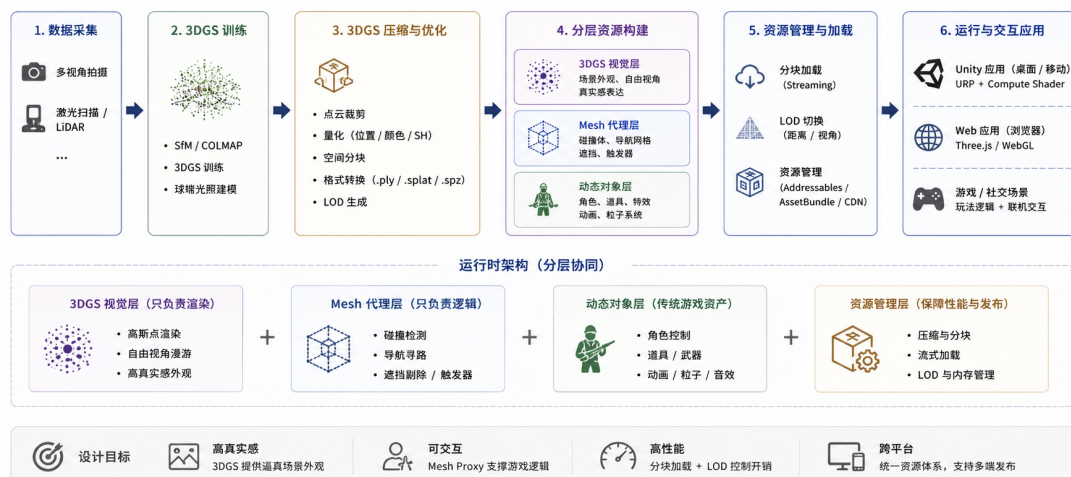


图 4: 3DGS 负责视觉表达，Mesh Proxy 负责碰撞和游戏逻辑，动态对象继续使用传统游戏资产。

6.2 阶段一：静态场景展示

第一阶段的目标是形成稳定可展示的 3DGS 场景浏览器，重点验证场景质量、加载速度、运行帧率和部署方式，而不追求复杂游戏交互。在该阶段，输入可以来自多视角图像、公开数据集或已有 .splat 场景；处理过程主要包括 3DGS 训练、格式转换、压缩和简单裁剪；输出形式可以是 Web 展示页面，也可以是 Unity 桌面端 Demo。

从产品形态上看，该阶段适合用于技术主页、作品展示、虚拟展厅、真实空间漫游和短视频虚拟背景预览等场景。其价值在于以较低交互复杂度验证 3DGS 的基础展示能力，并快速判断浏览器端和桌面端在视觉质量、帧率和加载体验上的差异。当前已经完成的 Web 端展示和 Unity Editor 渲染，均可以归入这一阶段。

6.3 阶段二：半动态游戏环境

第二阶段的目标是在静态 3DGS 背景上叠加角色控制、碰撞检测和基础玩法逻辑。该阶段不要求 3DGS 本身具备物理属性，而是采用“3DGS 视觉层 + Mesh Collider 层”的方式进行组合：3DGS 场景作为不可交互或弱交互的视觉背景，低模 Mesh 代理用于表示地面、墙体、楼梯、边界和触发区域，角色移动则由 Unity CharacterController、NavMesh，或 Web 端的 Rapier / Ammo 等物理库承担。这种结构可以形成一个可玩的空间漫游 Demo。用户看到的是具有真实感的 3DGS 场景，但实际的碰撞、导航和触发逻辑由代理 Mesh 完成。动态物体、UI、粒子效果、交互道具和游戏规则仍使用传统游戏资产实现。与直接尝试动态高斯更新相比，这一路线复杂度更低，工程可控性更强，也更适合作为入职作业阶段的主要展示形式。

6.4 阶段三：Web 原生轻量化产品

如果目标平台是 Web 页面、活动页、小游戏或抖音生态中的轻互动页面，则不建议优先将完整 Unity 工程导出为 WebGL。更合理的方式是采用 Web 原生 3DGS 渲染路线，将 3DGS 展示、基础相机控制和少量交互逻辑直接部署在 Three.js / WebGL 环境中。这样可以避免 Unity URP 带来的 Shader Variant 膨胀问题，也更容易与前端页面、账号系统、分享链路、数据埋点和运营活动结合。

该阶段可以围绕“真实地点空间化展示”“创作者 3D 房间”“虚拟展厅”“小游戏背景生成”和“短视频互动场景”展开产品实验。对于轻交互场景而言，Three.js 的渲染能力和前端生态已经能够支撑基本体验，而 Unity 更适合作为桌面端验证、编辑器调试和复杂交互原型的平台。

6.5 阶段四：长期能力建设

长期来看，3DGS 在游戏中的进一步应用还需要解决动态场景表达、光照解耦、编辑器集成、资产标准化和 AIGC 生成链路等问题。例如，动态 3DGS 可以支持时变场景和角色表达，光照解耦可以增强与游戏引擎 PBR 管线的兼容性，编辑器集成可以使 3DGS 从“查看器资产”转变为“可管理资产”，而 AIGC 生成链路则可能进一步降低场景生产门槛。

不过，这些方向具有较强研究属性和工程复杂度，并不适合作为当前入职作业的主要交付。更合理的策略是：近期交付一个可运行 Demo、一套清晰的工程边界判断，以及一份面向产品化应用的技术路线图；长期再围绕动态表达、工具链建设和跨平台部署逐步推进。

7 产品化价值与抖音场景落地点

7.1 从技术能力到产品价值

对于 AI 技术产品实习生而言，3DGS 调研不应停留在“技术是否先进”，而应回答它能否降低生产成本、提升用户体验或创造新的互动形式。结合当前实践，3DGS 的产品价值主要体现在三个方面：**降低真实空间数字化门槛**：用户或创作者通过手机拍摄即可生成可漫游场景；**提升内容表现力**：相比普通图片/视频背景，3DGS 支持自由视角和空间探索；**扩展游戏开发工具链**：为小游戏或 UGC 开发者提供快速生成静态背景和空间资产的能力。

7.2 短视频虚拟背景

短视频虚拟背景是近期最适合落地的方向之一。创作者可以通过拍摄房间、店铺、校园角落或旅行地点生成 3DGS 场景，再将其作为自由视角背景用于视频创作。相比传统绿幕或 2D 背景，3DGS 可以提供更强的空间感和镜头运动自由度。

该方向的交互需求较低，主要挑战是移动端训练/渲染成本、模板化编辑和上传压缩。因此可作为 P0 级产品探索方向。

7.3 虚拟展厅与空间社交

3DGS 适合表达真实空间，因此可以用于虚拟展厅、校园导览、品牌空间和线上活动场景。结合 Avatar、弹幕、语音或轻量互动，用户可以在真实重建空间中进行社交。对于抖音社交与互动业务，这类方向可以拓宽用户打开 App 的理由，从内容观看延伸到空间参与。

7.4 小游戏场景生成

小游戏开发者经常缺少高质量 3D 场景资产。若能通过 AIGC + 3DGS 构建“文本/图片/视频输入 → 场景生成 → 一键导入小游戏模板”的工具链，将显著降低开发门槛。不过该方向对交互、包体、移动端性能和编辑器集成要求更高，应作为 P1/P2 方向逐步推进。

7.5 应用优先级矩阵

表 6: 3DGS 在抖音社交与互动场景中的产品化优先级

应用场景	技术难度	包体压力	交互需求	优先级	说明
短视频虚拟背景	2/5	低	低	P0	视觉价值明显，交互链路短
虚拟展厅/空间漫游	3/5	中	中	P1	适合活动页和品牌空间
网页轻互动小游戏	3.5/5	中到高	中	P1	需控制包体和移动端性能
UGC 场景生成工具	4/5	高	中到高	P1/P2	需要工具链和模板体系
完整 3DGS 游戏	5/5	极高	极高	P2	当前不建议作为近期目标

8 交付

8.1 近期交付

近期最合适的交付不是训练大型 4D 世界模型，也不是强行做完整 Unity 游戏，而是形成一个清晰的“技术边界验证包”：一个 Web 端 3DGS 漫游页面；一个 Unity Editor 本地运行 Demo；一个 Three.js 轻游戏交互 Demo；一份产品化方案，说明短视频虚拟背景、虚拟展厅和小游戏场景生成的优先级。

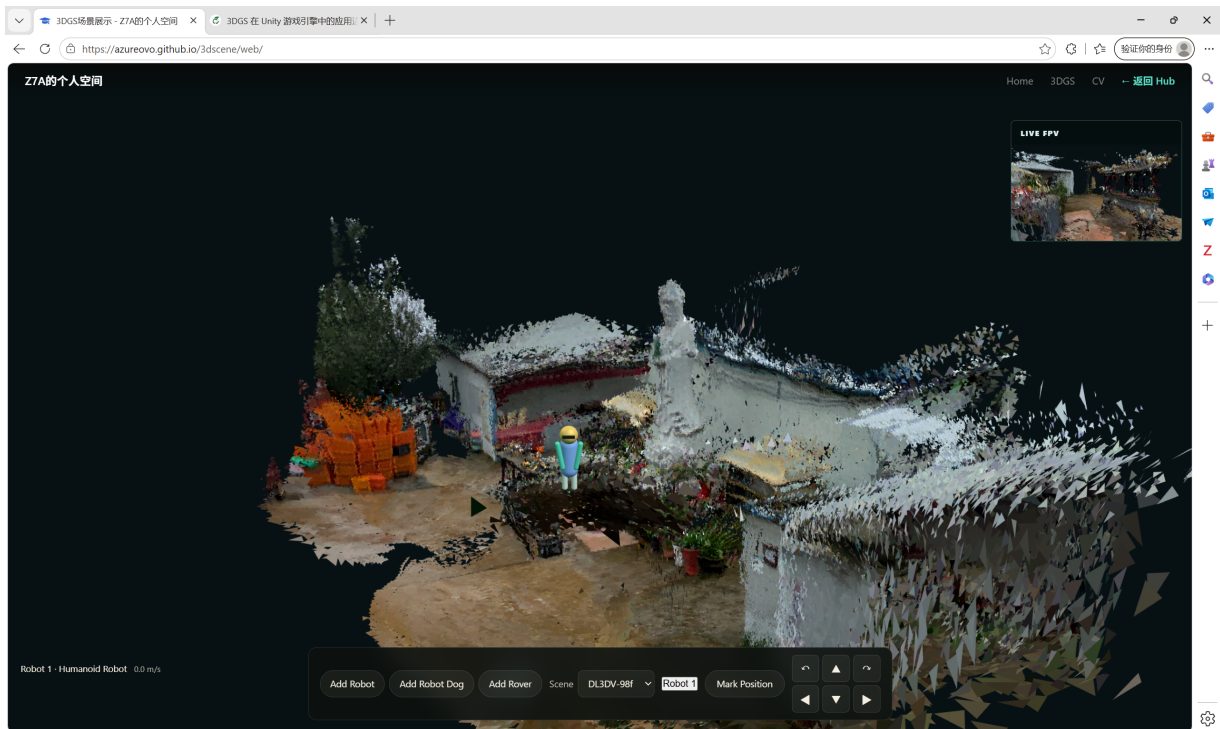


图 5: 呈现 3DGS 在网页端的交互。



图 6: 呈现 3DGS 在 Unity 情景的效果。

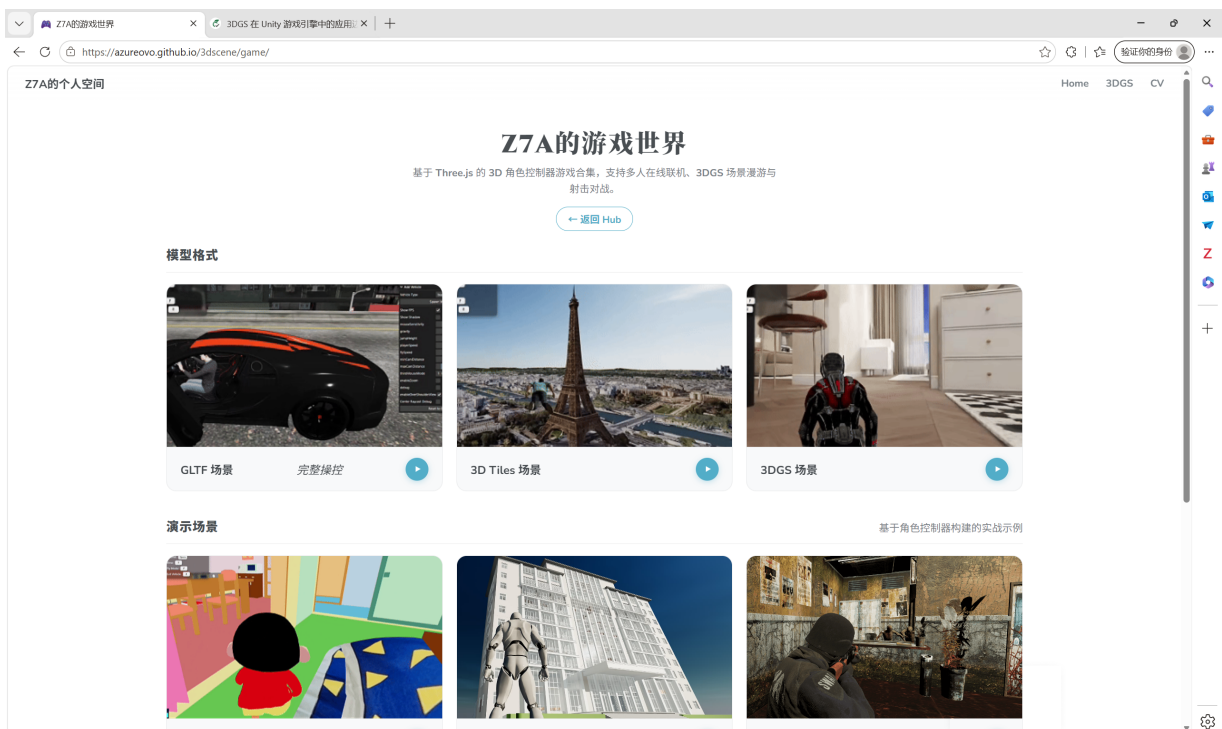


图 7: 呈现 3DGS 利用 three.js 呈现游戏效果。

9 风险分析与后续研究

9.1 主要风险

表 7: 风险分析与缓解策略

风险	表现	缓解策略
技术路线过重	试图直接做 4DGS 或完整 Unity 游戏，导致周期失控	将近期目标限定为静态视觉代理和轻交互 Demo
WebGL 构建不可控	Unity WebGL 打包失败或包体过大	转向 Web 原生 3DGS 渲染器, Unity 仅用于本地演示
产品价值不清晰	只展示技术炫酷，难对应业务需求	围绕短视频、虚拟空间和小游戏开发者工具组织价值
移动端性能不足	手机端帧率和内存不稳定	降低场景规模，采用压缩、分块和渐进式加载
交互能力不足	3DGS 只能看不能玩	使用 Mesh Collider 和传统 Mesh 动态物体补足交互

9.2 后续研究方向

后续可以沿四个方向推进：

- (1) **压缩与流送**：测试 SPZ、量化和分块加载策略，降低首屏体积；
- (2) **移动端适配**：评估 iOS/Android 浏览器和小程序环境中的 3DGS 渲染性能；
- (3) **编辑器工具**：实现简单的 Unity Importer、预览面板和 Collider Proxy 生成工具；
- (4) **光照解耦**：结合逆向渲染方法探索 3DGS 与动态光照的兼容性。

10 结论

本文围绕 3DGS 在 Unity 游戏引擎中的应用边界进行了调研与实践验证。通过 Web 端实时展示、Unity 桌面端渲染、Three.js 游戏环境，可以得到如下结论：

- (1) 3DGS 在静态复杂场景的视觉表达上具有明确价值，适合承担远景、背景、虚拟展厅、空间漫游和加载关卡视觉代理；
- (2) Unity Editor 桌面端 3DGS 渲染性能基本达标，但 Unity WebGL 由于 Shader 变体膨胀和 Compute Shader 后端限制，暂不适合作为近期线上化主路径；
- (3) 3DGS 不应被定位为 Mesh 的完整替代，而应采用“3DGS 视觉层 + Mesh 碰撞代理 + 传统动态对象”的混合架构；

- (4) 面向抖音社交与互动场景,短视频虚拟背景、虚拟展厅和轻量小游戏场景生成是比完整 3DGS 游戏更现实的落地方向;

总体而言,3DGS 在游戏中的近期价值是提供一种高真实感、低拓扑约束的静态视觉代理。它可以帮助开发者更快地把真实空间或复杂背景带入交互应用,但仍需要与传统 Mesh 管线协同工作。只有在明确边界的基础上推进工程验证,才能避免把前沿技术误用为泛化解决方案。